

JAX / Gymnasium 強化學習專案開發計畫書

面向 AI Agent 自動化構建之落地執行規格與架構藍圖

目標硬體

NVIDIA RTX 3070 Ti (8GB)

核心框架

JAX + Flax + Optax

測試環境

Gymnasium (CartPole-v1)

文件版本

v1.0.0 (Production)

1. 專案開發目標與硬體約束規約

本專案旨在建構一套基於 JAX / Flax 的高效能 Deep Q-Network (DQN) 強化學習系統。專案需針對 NVIDIA RTX 3070 Ti GPU (Ampere 架構, 8GB VRAM) 進行特定記憶體與編譯優化，以確保高吞吐量與低延遲訓練。

硬體與顯存管理規範

- 限制 XLA 顯存預分配率為 60% (`XLA_PYTHON_CLIENT_MEM_FRACTION=.60`)，防止 VRAM 溢出。
- 開啟 Float32 / Mixed Precision 計算模式，充分利用 Tensor Cores。

JAX 純函數與 JIT 編譯約束

- 訓練核心必須封裝為純函數 (Pure Functions)，並掛載 `@jax.jit`。
- 禁止在 JIT 函數內部調用 Python 側邊效應 (如 `print`、外部狀態變更)。

2. 專案標準目錄結構 (Repository Directory Layout)

AI Agent 必須嚴格按照以下架構創建目錄與模組檔案：

```
jax_rl_cartpole/  
├── config.py          # 系統參數、超參數與 GPU 相關環境變數設定  
├── models.py          # Flax Linen 神經網路架構定義 (QNetwork)  
├── replay_buffer.py    # 環形經驗回放池 (Vectorized Replay Buffer)  
├── agent.py           # JAX Pure Function 計算核心與 Optax 優化器邏輯  
├── train.py           # 主訓練迴圈、日誌輸出與 Checkpoint 保存  
├── evaluate.py        # 模型評估與 Gym 渲染驗證腳本  
└── requirements.txt    # 嚴格版本的依賴套件清單
```

3. 依賴套件配置規範 (requirements.txt)

套件名稱	指定版本 / 來源	用途說明	屬性
jax[cuda12_pip]	>=0.4.20 (via JAX Releases)	JAX CUDA 12 核心庫與 XLA 編譯器	核心
flax	>=0.7.5	高階神經網路模組設計 (Linen API)	核心
optax	>=0.1.7	JAX 專用梯度優化器組合庫	核心
gymnasium[classic-control]	>=0.29.1	標準強化學習環境介面	核心

4. 核心模組詳細實作規格

4.1 網路架構與 JAX 純函數 (models.py & agent.py)

Agent 實作時需嚴格遵循 Flax Linen 的結構設計，網路介面需滿足多維 Batch 輸入：

```
import flax.linen as nn
import jax
import jax.numpy as jnp

class QNetwork(nn.Module):
    action_dim: int

    @nn.compact
    def __call__(self, x):
        x = nn.Dense(features=128)(x)
        x = nn.relu(x)
        x = nn.Dense(features=128)(x)
        x = nn.relu(x)
        return nn.Dense(features=self.action_dim)(x)

@jax.jit
def train_step(params, target_params, opt_state, batch, gamma, model_apply_fn, opt_update_fn):
    states, actions, rewards, next_states, dones = batch

    next_q = model_apply_fn(target_params, next_states)
    max_next_q = jnp.max(next_q, axis=-1)
    target_q = rewards + (1.0 - dones) * gamma * max_next_q

    def loss_fn(p):
        q_vals = model_apply_fn(p, states)
        one_hot = jax.nn.one_hot(actions, q_vals.shape[-1])
        chosen_q = jnp.sum(q_vals * one_hot, axis=-1)
        return jnp.mean((chosen_q - target_q) ** 2)

    loss, grads = jax.value_and_grad(loss_fn)(params)
    updates, new_opt_state = opt_update_fn(grads, opt_state, params)
    new_params = optax.apply_updates(params, updates)
    return new_params, new_opt_state, loss
```

5. AI Agent 落地執行步驟 (Execution Roadmap)

階段	里程碑任務	產出物 / 交付標準	驗證方式
Phase 1	環境準備與 CUDA 綁定	完成 requirements.txt 安裝並驗證 GPU	jax.devices()[0].device_kind 需顯示 3070 Ti
Phase 2	模組代碼建構	完成所有 .py 檔案實作與單元測試	執行 python -m unittest 無異常
Phase 3	模型訓練與 JIT 驗證	執行 train.py 完成 200 Episodes 訓練	前 100 步平均 Reward 達到 > 475 分
Phase 4	推論與成果驗證	執行 evaluate.py 輸出評估結果與 Checkpoint	無 Epsilon 隨機探索下連續 10 次獲得 500 分

6. 驗證與 Acceptance Criteria (收穫標準)

最終交付檢查清單 (Checklist)
